

SOFTWARE REQUIREMENTS SPECIFICATION FOR

Implementation of RGB Dimensionality Reduction Techniques using Denoising Autoencoder

Prepared by

ANKITA A (231IT007)

NELLURI THANMAYA SRI (231IT043)

PETETI HANVITHA (231IT048)

DEPARTMENT OF INFORMATION TECHNOLOGY
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA SURATHKAL

Table of Contents

1. Introduction

1.1 Purpose.....	1
1.2 Scope.....	1
1.3 Definitions, Acronyms and Abbreviations.....	1
1.4 References.....	2
1.5 Overview.....	2

2.Overall Description

2.1 Product Perspective.....	2
2.2 Product Functions.....	2
2.3 User Classes.....	3
2.4 Constraints.....	3
2.5 Assumptions and Dependencies.....	3

3.Specific Requirements

3.1 Functional Requirements.....	4
3.2 External Interface Requirements	
3.2.1 User Interfaces.....	5
3.2.2 Hardware Interfaces.....	5
3.2.3 Software Interfaces.....	6
3.2.4 Communications Interfaces.....	6
3.3 Other Non-Functional Requirements.....	6
3.4 Constraints.....	7

Appendix A.....	8
------------------------	----------

Index.....	9
-------------------	----------

1. Introduction

1.1 Purpose

The purpose of this project is to develop a web-based RGB image dimensionality reduction using a Denoising Autoencoder. The system is designed to enable users to preprocess, denoise, and compress RGB images, while preserving essential features for further analysis. The software provides an intuitive graphical user interface for uploading images, and viewing results in real time. This platform is intended to support students, researchers, and developers who are exploring image processing and machine learning concepts.

1.2 Scope

The project focuses on automating the training, validation, and testing of image denoising models through an intuitive frontend and backend system. Users can upload noisy images, and retrieve denoised outputs directly via the interface. The current scope includes dataset handling, model training, inference, and result visualization. Future extensions may include:

- Supporting larger datasets and multiple model architectures.
- Permanent storage of results and user profiles using a database.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
API	Application Programming Interface
DAE	Denoising Autoencoder
Dimensionality Reduction	Process of reducing the number of features while preserving essential information
Latent Space / Latent Vector	A compressed feature representation of the input data, typically used in deep learning models
MSE	Mean Squared Error
PSNR	Peak Signal-to-Noise Ratio
RGB	Red,Green,Blue colour model.
SSIM	Structural Similarity Index Measure

1.4 References

[1] Vincent, P., et al. "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion." *JMLR*, 2010.

[2] Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. MIT Press, 2016.

[3] NumPy, *NumPy Documentation*. [Online]. Available: <https://numpy.org/doc/stable>. [Accessed: Aug. 23, 2025].

[4] Keras, *Keras: The Python Deep Learning API*. [Online]. Available: <https://keras.io>. [Accessed: Aug. 24, 2025].

1.5 Overview

The following sections of this document present a detailed description of the system requirements, including both functional and non-functional aspects. It also outlines the interfaces, performance needs, design constraints, and safety considerations that guide the development process. Additional sections provide supporting information such as definitions and models that ensure clarity and completeness of the specification.

2. Overall Description

2.1 Product Perspective

The Image Denoising Autoencoder is a stand-alone software application built for image processing and evaluation. The system uses deep learning to remove noise from corrupted images and restore them to their near-original form. The product functions as a research and experimental platform for enhancing visual quality and can be integrated into larger applications such as medical imaging systems, satellite image processing, and photography enhancement tools.

It is a data-driven application dependent on training datasets such as BSDS500 and supports evaluation through performance metrics including PSNR, SSIM, and MSE. The system requires compatible GPU hardware (e.g., NVIDIA CUDA-enabled GPUs) and relies on Python, TensorFlow for model implementation.

2.2 Product Functions

Dataset Handling

- Import dataset and inject noise.
- Perform preprocessing such as resizing, normalization and augmentation.

Model Training

- Train the autoencoder using noisy input and clean target pairs.
- Save and load trained model weights.

Image Denoising

- Accept noisy input images.
- Apply the trained model to generate denoised images.

Performance Evaluation

- Compute PSNR (Peak Signal-to-Noise Ratio) to assess image clarity.
- Compute SSIM (Structural Similarity Index) to evaluate structural similarity.
- Compute MSE (Mean Squared Error) to measure pixel-wise error.

Visualization & Reporting

- Display noisy, denoised, and original images side by side.
- Generate evaluation reports with computed metrics.

2.3 User Classes

- **Researchers/Developers:** Use the system for experimenting with different noise types and denoising architectures.
- **Students/Learners:** Use the software as a learning tool for understanding autoencoders, CNNs, and image quality metrics.
- **Domain Professionals** (e.g., radiologists, satellite image analysts, photographers): Use the denoising outputs in real-world scenarios like enhancing medical scans, astronomical data, or digital photography.

2.4 Constraints

- **C.1 Regulatory Policies** – Must comply with open-source licensing of datasets (e.g., BSDS500, ImageNet subsets).
- **C.2 Hardware Constraints** – Requires GPU acceleration for large-scale training (minimum 8GB VRAM recommended).
- **C.3 Software Interfaces** – Dependent on TensorFlow, NumPy, scikit-learn. and Matplotlib for training, evaluation, and visualization.
- **C.4 Parallel Operation** – Supports multi-GPU training if CUDA is available; otherwise defaults to CPU with limited efficiency.
- **C.5 Audit Functions** – Logs all training sessions, hyperparameters, and evaluation metrics for reproducibility.
- **C.6 Control Functions** – Requires checkpoint saving, early stopping, and validation split for stable training.
- **C.7 Criticality of the Application** – In real-world domains such as medical imaging, reliability is critical, as erroneous denoising can lead to misinterpretation.

2.5 Assumptions and Dependencies

- The system assumes the availability of a labeled dataset containing pairs of clean and noisy images.
- It is assumed that users have access to compatible OS platforms (Linux/Windows with Python version 3.8 or higher).
- The training performance assumes availability of a CUDA-enabled GPU; without it, execution time will increase drastically.
- Dependency on third-party libraries (TensorFlow, NumPy, Matplotlib) being installed and up-to-date.
- Internet connectivity is assumed if datasets need to be downloaded dynamically.
- If the dataset distribution changes (e.g., licensing issues), the product requirements must be updated to accommodate alternate datasets.

3. Specific Requirements

3.1 Functional Requirements

F.1 Upload RGB image datasets

Description: Users can upload RGB image datasets in common formats (JPEG, PNG) via the interface for model training and evaluation.

Input: RGB image dataset files uploaded through the UI.

Output: Confirmation of successful upload.

F.2 Noise injection to the uploaded images.

Description: The system shall generate noisy versions of clean images by adding gaussian noise.

Input: Clean RGB images.

Output: Noisy images paired with their clean counterparts for training.

F.3 Apply Random Cropping and Augmentation

Description: The system shall perform random cropping to create uniform input sizes across images, ensuring spatial features are preserved without deformation.

Input: Dataset images of varying dimensions.

Output: Cropped images of fixed input dimensions fed into the model.

F.4 Encode Noisy Images to Latent Space

Description: The system shall process noisy images through the encoder to produce a compressed latent representation.

Input: Noisy RGB images.

Output: Latent vectors capturing essential features.

F.5 Decode Latent Vectors to Reconstruct Images

Description: The system shall decode latent representations to reconstruct clean RGB images.

Input: Latent vectors.

Output: Reconstructed (denoised) RGB images.

F.6 Compute Reconstruction Loss

Description: The system shall calculate reconstruction loss (e.g., mean squared error) between the original clean images and their reconstructions for training.

Input: Clean images and reconstructed outputs.

Output: Loss values used to guide training optimization.

F.7 Train Model Iteratively

Description: The system shall train the autoencoder in minibatches iterating through data augmentation, encoding/decoding, loss calculation, and weight updates.

Input: Noisy-clean image pairs, model architecture, hyperparameters (learning rate, epochs, batch size).

Output: Updated model weights and training logs (loss per epoch).

F.8 Model Evaluation

Description: The system shall evaluate the trained model using quantitative metrics:

- **Mean Squared Error (MSE):** To measure pixel-wise error between original and reconstructed images.
- **Peak Signal-to-Noise Ratio (PSNR):** To measure reconstruction quality relative to original images.
- **Structural Similarity Index (SSIM):** To assess structural similarity between images.
- The system shall generate training logs (loss per epoch, evaluation metrics).

Input:

- Trained model weights.
- Validation/test dataset (images).

Output:

- Quantitative metrics (MSE, PSNR, SSIM) in numerical format.
- Training and evaluation logs (loss curves, metric values per epoch).
- Visualization outputs (optional graphs of loss/metrics).

F.9 Save and Load Trained Model

Description: The system shall allow saving trained model architecture and weights, and loading them later for inference or further training.

Input: Save command.

Output: Confirmation message.

F.10 Perform Inference on New Images

Description: The system shall denoise the input image using the trained encoder-decoder pipeline and present the reconstructed output.

Input: New RGB image.

Output: Denoised RGB image.

3.2 External Interface Requirements

3.2.1 User Interfaces

Description:

A graphical user interface (GUI) dashboard shall be provided for dataset upload, model training, and result visualization. The GUI shall also provide graphical previews of noisy vs. denoised images.

Input: User uploads dataset files.

Output:

- Confirmation of dataset upload.
- Training progress and results visualization.
- Side-by-side comparison of noisy and denoised images.

3.2.2 Hardware Interfaces

Description:

The system shall be compatible with both CPU and GPU-enabled machines. For efficient training, NVIDIA CUDA-supported GPUs shall be utilized.

3.2.3 Software Interfaces

Description:

The system shall interface with machine learning libraries and data processing tools for model implementation and visualization. It shall interact with the local file system for storing datasets, checkpoints, and results.

Input:

- Libraries: TensorFlow, NumPy, Matplotlib, scikit-learn.
- File system inputs (uploaded datasets).

Output:

- Model training, logs, and visualization outputs.

3.2.4 Communications Interfaces

Description:

The system shall provide communication between the frontend and backend through RESTful APIs over HTTP/HTTPS. All requests and responses shall use JSON format for parameters, while image data shall be exchanged via file uploads. Authentication shall be enforced for secure access.

Input:

API requests including dataset uploads.

Output:

API responses containing denoised images, training metrics (PSNR, SSIM, loss).

3.3 Other Non-Functional Requirements

NF.1 Platform Requirements

Description:

The system shall be platform independent and run seamlessly on both Windows and UNIX/Linux environments. This ensures wider accessibility for developers, testers, and end-users across diverse computing infrastructures.

NF.2 Web Support

Description:

The system shall be accessible via a standard web browser without requiring local installation of additional software. All core functionalities such as dataset upload, training initiation, and output visualization shall be invoked remotely over the web.

NF.3 Secure API Access

Description:

The system shall require authentication via API keys for all API interactions to prevent unauthorized usage and ensure secure communication.

3.4 Design Constraints

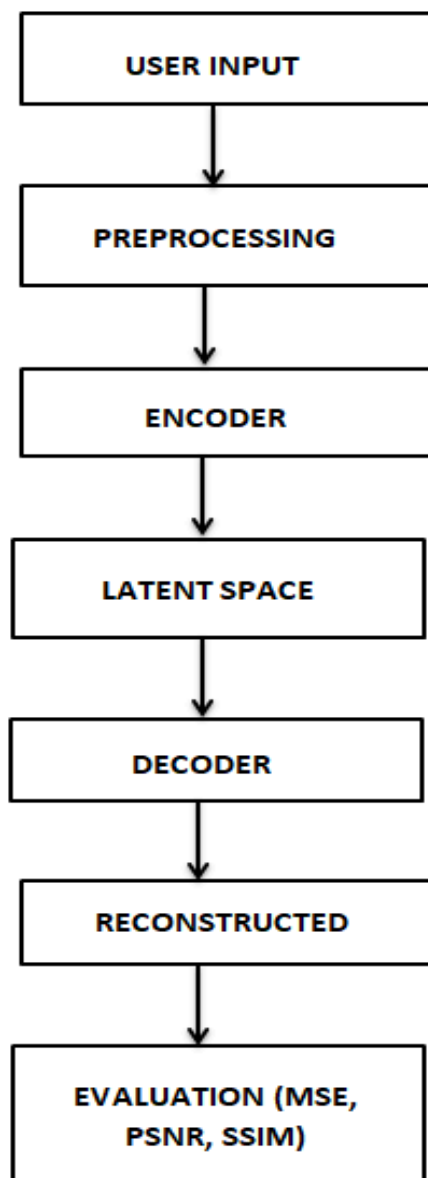
Hardware Limitations: The system is designed to run efficiently on machines with at least 8GB RAM and a GPU with CUDA support for model training. CPU-only environments will be supported but with reduced performance.

Framework Dependency: The backend model training must use TensorFlow, and switching frameworks is out of scope for the current design.

Appendix A

A.1 Block Diagram

The block diagram below illustrates the overall workflow of the proposed system. The process begins with the user uploading an input image, which is then preprocessed to ensure it is suitable for model processing. The encoder component of the denoising autoencoder compresses the image into a latent vector representation. This latent space captures the essential features of the input while discarding noise. The decoder reconstructs the image from the latent vector, producing a denoised or restored version. Finally, evaluation metrics such as MSE, PSNR, and SSIM are calculated to assess the quality of reconstruction.



Index

A

[API](#) – Application Programming Interface

C

[CUDA](#) – Compute Unified Device Architecture (GPU acceleration)

D

[DAE](#) – Denoising Autoencoder

[Dimensionality Reduction](#) – Reducing features while preserving essential info

[Dataset Handling](#) – Importing and preprocessing noisy/clean image datasets

G

[Gaussian Noise](#) – Random noise added to images during training

[GPU](#) – Graphics Processing Unit (required for efficient training)

[GUI](#) – Graphical User Interface for uploading images and visualizing results

L

[Latent Space / Latent Vector](#) – Compressed representation of input data

M

[Model Training](#) – Iterative optimization of encoder-decoder weights

[MSE](#) – Mean Squared Error (evaluation metric)

N

[Noise Injection](#) – Process of adding synthetic noise to clean images

P

[PSNR](#) – Peak Signal-to-Noise Ratio (image quality metric)

S

[SSIM](#) – Structural Similarity Index Measure

Version

Version	Last updated	Reason for change
1.0	12-08-2025	Initial Update
2.0	24-08-2025	Major reviews done